Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251229**
Nama Lengkap	**Ignatius Harya Nugraha**
Minggu ke / Materi	**04 / Percabangan**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Boolean Expression dan Logical Operator

Apa Itu Boolean Expression?

Pernahkah kalian mendapat notifikasi "Selamat! Kamu dapat gratis ongkir karena belanja minimal Rp 50.000"? Ternyata dibalik notif itu, ada yang namanya boolean expression.

Boolean expression adalah pertanyaan yang jawabannya cuma ada dua: YA atau TIDAK. Dalam dunia pemrograman, jawaban itu diwakili dengan tipe data Boolean yang memiliki nilai True (benar) dan False (salah).

Dalam kehidupan kita sehari-hari Boolean Expression bisa saja terjadi contohnya adalah sebagai berikut:

- "Apakah uang saku kamu cukup buat beli makan siang?" → True atau False
- "Apakah kamu sudah mengerjakan tugas?" → True atau False
- "Apakah umur kamu sudah 17 tahun untuk bisa bikin KTP?" → True atau False

Operator Perbandingan

Untuk membuat boolean expression, kita memakai operator perbandingan. Operator perbandingan ini seperti cara kita bertanya dalam bahasa Indonesia, tapi dalam bahasa pemrograman:

OPERATOR	Arti	Contoh
x == y	Apakah x sama dengan y?	nilai == 100
x != y	Apakah x beda dengan y?	status != "lulus"
x > y	Apakah x lebih besar dari y?	umur > 17
x >= y	Apakah x lebih besar atau sama?	ipk >= 2.75
x < y	Apakah x lebih kecil dari y?	suhu < 37
x <= y	Apakah x lebih kecil atau sama?	nilai <= 100

Salah satu kesalahan yang sering terjadi pada pemula adalah kebingungan antara tanda = dan ==. Tanda = digunakan untuk memberikan nilai kepada variabel, sedangkan == digunakan untuk membandingkan dua nilai. Perbedaan ini sangat penting karena kesalahan penggunaan dapat menyebabkan error atau logika program yang keliru. Pemahaman yang tepat terhadap operator

perbandingan akan membantu mahasiswa membangun kondisi yang akurat dan sesuai dengan tujuan program.

Operator perbandingan tidak hanya digunakan untuk angka, tetapi juga dapat diterapkan pada tipe data lain. Yang menunjukkan fleksibilitas Boolean Expression dalam menangani berbagai jenis data, sehingga program dapat menjadi lebih dinamis.

Struktur Percabangan

Struktur percabangan paling dasar dalam if. Struktur ini digunakan ketika program perlu menjalankan suatu perintah hanya jika kondisi tertentu terpenuhi. Jika kondisi bernilai True, maka blok kode di bawahnya akan dieksekusi. Jika bernilai False, blok tersebut akan dilewati tanpa menimbulkan kesalahan.

Penggunaan if menjadi sangat efektif ketika hanya ada satu kondisi utama yang ingin diuji. Namun dalam banyak kasus, program memerlukan dua kemungkinan hasil. Dalam situasi tersebut digunakan struktur if-else. Struktur ini memaksa program untuk memilih satu dari dua jalur yang tersedia. Ketika kondisi terpenuhi, jalur pertama dijalankan; ketika tidak terpenuhi, jalur alternatif akan dijalankan.

Namun apabila terdapat lebih dari dua kemungkinan, digunakan struktur if-elif-else. Struktur ini memungkinkan evaluasi beberapa kondisi secara berurutan. Program akan membaca kondisi dari atas ke bawah dan mengeksekusi blok kode pertama yang bernilai True. Karena itu kita perlu untuk Menyusun logika urutannya agar program dapat berjalan sesuai dengan keinginan kita.

Contoh penulisan struktur if, if-else, if-elif-else:

```
nilai = 78

if nilai >= 60:
    print("Selamat, kamu lulus!")

if nilai >= 60:
    print("Lulus")
else:
    print("Tidak Lulus")

if nilai >= 85:
    print("Grade A")
elif nilai >= 75:
    print("Grade B")
elif nilai >= 65:
    print("Grade C")
else:
    print("Grade D/E")
```

Operator Logika dan Kompleksitas Kondisi

Dalam penerapan suatu logika dalam pemrograman satu kondisi sering kali tidak cukup untuk mendapatkan hasil yang akurat. Oleh karena itu, terdapat operator logika seperti and, or, dan not. Operator ini memungkinkan penggabungan beberapa kondisi menjadi satu ekspresi yang lebih kompleks.

Operator and berarti semua kondisi bernilai True agar hasil akhirnya True. Operator ini sering digunakan dalam situasi yang membutuhkan beberapa persyaratan sekaligus. Sebaliknya, operator or hanya membutuhkan satu kondisi bernilai True agar keseluruhan ekspresi bernilai True. Operator ini cocok digunakan dalam situasi yang memiliki beberapa alternatif pemenuhan syarat. Sementara itu, operator not digunakan untuk membalik nilai kebenaran suatu kondisi.

Contoh penggunaan operator logika pada python:

```
umur = 20
punya_ktp = True
nilai_ujian = 72

if umur >= 17 and punya_ktp:
    print("Bisa mendaftar layanan")

if nilai_ujian >= 75 or nilai_ujian >= 60:
    print("Boleh mengikuti remedial")

if not punya_ktp:
    print("Harap buat KTP terlebih dahulu")
```

Penggunaan operator logika pada pemrograman bukan hanya soal memeriksa satu kondisi tunggal, melainkan sering kali melibatkan kombinasi berbagai hal yang saling berkaitan. Kemampuan menyusun kondisi kompleks dengan operator logika menjadi indikator pemahaman logika yang lebih matang.

If-Elif-Else pada Ternary If

Struktur if-elif-else merupakan perluasan dari if-else yang memungkinkan pengecekan beberapa kondisi secara berurutan. Kata kunci elif adalah singkatan dari "else if", artinya kondisi ini hanya akan diperiksa jika kondisi-kondisi sebelumnya bernilai False. Dengan demikian, hanya satu blok kode yang akan dieksekusi dalam satu kali jalannya program.

Ternary if, atau disebut juga conditional expression, adalah cara singkat untuk menuliskan if-else dalam satu baris. Sintaksnya adalah: nilai_jika_true if kondisi else nilai_jika_false. Ternary if berguna ketika kita ingin menetapkan nilai suatu variabel berdasarkan kondisi tertentu secara ringkas. Namun untuk kondisi yang kompleks, tetap disarankan menggunakan if-else biasa agar kode lebih mudah dibaca dan dipahami.

Contoh penerapan Ternary If dalam python:

```
nilai = 68
status = "A" if nilai >= 80 else “B” if nilai >= 70 else "C"
print(f"Nilai dalam huruf: {status}")
```

Nested If dan Struktur Keputusan Bertingkat

Nested if (percabangan bersarang) adalah penggunaan struktur if di dalam fungsi if lainnya. Struktur ini digunakan ketika keputusan yang perlu diambil terdiri dari beberapa tahap yang saling bergantung satu sama lain. Kondisi di dalam hanya akan diperiksa apabila kondisi di luarnya sudah terpenuhi terlebih dahulu. Hal ini memberikan ruang untuk program memodelkan proses pengambilan keputusan yang lebih mendetail dan hierarkis.

Meskipun nested if sangat berguna, penggunaannya yang terlalu dalam dapat membuat kode menjadi sulit dibaca dan dipahami. Ini sering disebut sebagai "pyramid of doom" karena bentuk indentasi kode yang semakin menjorok ke kanan. Umumnya disarankan untuk tidak membuat percabangan lebih dari tiga tingkat. Jika diperlukan, pertimbangkan untuk memecah logika menjadi beberapa fungsi terpisah agar kode tetap bersih dan mudah dipelihara.

Berikut adalah contoh kode dalam penerapan nested if dalam system pendaftaran beasiswa berdasarkan keaktifan, penghasilan orang tua, dan IPK:

```
ipk = float(input("Masukkan IPK kamu: "))
penghasilan_ortu = int(input("Penghasilan orang tua (Rp/bulan): "))
aktif_organisasi = input("Aktif di organisasi? (ya/tidak): ")

if ipk >= 3.0:
    print("IPK memenuhi syarat.")
    if penghasilan_ortu <= 3000000:
        print("Penghasilan orang tua memenuhi syarat.")
        if aktif_organisasi == "ya":
            print("Selamat! Kamu lolos seleksi beasiswa penuh.")
        else:
            print("Kamu lolos beasiswa parsial (tidak aktif organisasi).")
    else:
        print("Maaf, penghasilan orang tua melebihi batas.")
else:
print("Maaf, IPK kamu belum memenuhi syarat minimum 3.0.")
```

Pada contoh di atas, program pertama-tama memeriksa apakah IPK mahasiswa memenuhi syarat. Jika ya, baru diperiksa kondisi penghasilan orang tua. Jika itu pun terpenuhi, barulah status keaktifan organisasi diperiksa untuk menentukan jenis beasiswa yang didapatkan. Ini adalah contoh pengambilan keputusan bertingkat yang mencerminkan alur seleksi nyata.

Error Handling menggunakan Try dan Except

Error handling adalah mekanisme dalam pemrograman yang digunakan untuk menangani kesalahan (error) yang terjadi saat program berjalan. Tanpa error handling, sebuah error sekecil apapun dapat menyebabkan program berhenti secara tiba-tiba dan tidak terkendali. Dalam

sebuah program, error dibedakan menjadi dua jenis utama: Syntax Error dan Runtime Error (Exception).

Syntax Error terjadi saat kode tidak ditulis sesuai aturan bahasa Program, sehingga program tidak bisa berjalan sama sekali. Sementara Exception terjadi saat program sudah berjalan tetapi menemui situasi yang tidak bisa ditangani, seperti membagi angka dengan nol, mengakses index di luar batas list, atau memasukkan tipe data yang tidak sesuai. Beberapa jenis exception yang sering ditemui:

- ValueError – terjadi saat fungsi menerima nilai yang tidak sesuai, misalnya memasukkan huruf saat program meminta angka.
- ZeroDivisionError – terjadi saat terjadi pembagian dengan angka nol.
- TypeError – terjadi saat operasi diterapkan pada tipe data yang tidak sesuai.
- IndexError – terjadi saat mengakses indeks list yang berada di luar batas.
- FileNotFoundError – terjadi saat file yang ingin dibuka tidak ditemukan di sistem.

Struktur try-except adalah cara Python menangani error secara terstruktur dan terkontrol. Blok try berisi kode yang berpotensi menimbulkan error. Jika error terjadi di dalam blok try, program tidak langsung berhenti, melainkan akan melompat ke blok except yang sesuai untuk menangani error tersebut. Ini memungkinkan program untuk tetap berjalan dan memberikan respons yang informatif kepada pengguna, bukan sekadar pesan error yang membingungkan.

Struktur lengkap try-except juga dapat digabung dengan if else. Perintah else akan dijalankan hanya jika tidak ada error yang terjadi pada blok try, berguna untuk kode lanjutan yang hanya relevan jika proses berhasil. Blok finally akan selalu dijalankan baik ada error maupun tidak. Blok finally umumnya digunakan untuk membersihkan resource, seperti menutup koneksi database atau file yang sudah dibuka, agar tidak terjadi kebocoran resource (resource leak).

BAGIAN 2: LATIHAN MANDIRI (60%)

Latihan 4.1

Implementasikan penanganan kesalahan input pengguna dari program-program pada Contoh 3.1, 3.2 dan 3.3.

```
#Contoh 4.1
inputuser = input("Masukkan suhu tubuh (derajat Celcius): ")
try:
    suhu = float(inputuser)
    if suhu >= 38:
        print("Anda demam")
    else:
        print("Anda tidak demam")
except ValueError:
    print("Input tidak valid! Masukkan angka suhu tubuh yang benar.")

#Contoh 4.2
try:
    bilangan = int(input("Masukkan suatu bilangan: "))
    hasil = "Positif" if bilangan > 0 else ("Negatif" if bilangan < 0 else "Nol")
    print(hasil)
except ValueError:
    print("Input tidak valid! Masukkan bilangan bulat.")

#Contoh 4.3
try:
    a = int(input("Masukkan bilangan pertama: "))
    b = int(input("Masukkan bilangan kedua: "))
    c = int(input("Masukkan bilangan ketiga: "))

    if a > b and a > c:
        print("Terbesar:", a)
    elif b > a and b > c:
        print("Terbesar:", b)
    elif c > a and c > b:
        print("Terbesar:", c)
    else:
        print("Ada dua atau lebih bilangan yang sama besar.")

except ValueError:
    print("Input tidak valid! Semua input harus berupa bilangan bulat.")
```

Latihan 4.2

Implementasikan percabangan pada Contoh 3.2 (Positif-Negatif) menggunakan ternary operator.

```python
try:
    bilangan = int(input("Masukkan suatu bilangan: "))
    hasil = "Positif" if bilangan > 0 else ("Negatif" if bilangan < 0 else "Nol")
    print(hasil)
except ValueError:
    print("Input tidak valid! Masukkan bilangan bulat.")
```

Latihan 4.3

Buatlah sebuah program yang dapat menampilkan jumlah hari dalam suatu bulan di tahun 2020. Program meminta pengguna memasukkan nomor bulan (1-12), kemudian program akan menampilkan jumlah hari pada bulan tersebut. Sebagai contoh, perhatikan input dan output berikut ini:

Masukkan bulan (1-12): 7

Jumlah hari: 31

Lengkapi program tersebut dengan penanganan kesalahan jika pengguna memasukkan bulan yang salah. Penanganan kesalahan dalam bentuk memunculkan pesan bahwa bulan yang diinputkan oleh pengguna tersebut tidak valid.

```python
try:
    bulan = int(input("Masukkan bulan (1-12): "))

    if bulan < 1 or bulan > 12:
        print("Bulan yang Anda masukkan tidak valid.")
        print("Masukkan nomor bulan antara 1 sampai 12.")
    elif bulan == 2:
        print("Jumlah hari: 29")
    elif bulan > 2 and bulan % 2 == 0:
        print("Jumlah hari: 30")
    else:
        print("Jumlah hari: 31")

except ValueError:
    print("Input tidak valid! Masukkan angka nomor bulan (1-12).")
```

Latihan 4.4

Sebuah program meminta pengguna memasukkan ketiga panjang sisi suatu segitiga (berarti pengguna memasukkan tiga bilangan). Jika ketiga sisi segitiga tersebut semuanya sama, tampilkan pesan: "3 sisi sama". Jika hanya ada dua sisi yang sama panjang, tampilkan pesan "2 sisi sama". Jika tidak ada yang sama maka tampilkan pesan: "Tidak ada yang sama". Sebagai contoh, perhatikan input dan output berikut ini:

```
Masukkan sisi 1: 14
Masukkan sisi 2: 18
Masukkan sisi 3: 11
Tidak ada yang sama

Masukkan sisi 1: 22
Masukkan sisi 2: 22
Masukkan sisi 3: 22
3 sisi sama

Masukkan sisi 1: 8
Masukkan sisi 2: 9
Masukkan sisi 3: 8
2 sisi sama
```

Lengkapi program tersebut dengan penanganan kesalahan jika pengguna memasukkan input yang tidak valid.

```python
try:
    sisi1 = float(input("Masukkan sisi 1: "))
    sisi2 = float(input("Masukkan sisi 2: "))
    sisi3 = float(input("Masukkan sisi 3: "))

    if sisi1 <= 0 or sisi2 <= 0 or sisi3 <= 0:
        print("Input tidak valid! Panjang sisi harus bernilai positif.")
    elif sisi1 == sisi2 == sisi3:
        print("3 sisi sama")
    elif sisi1 == sisi2 or sisi2 == sisi3 or sisi1 == sisi3:
        print("2 sisi sama")
    else:
        print("Tidak ada yang sama")

except ValueError:
    print("Input tidak valid! Masukkan angka untuk panjang sisi segitiga.")
```

Link Repository Github : https://github.com/Haigry/PrakAlPro-71251229